request gets its own scoped container, so concurrent requests to the same resource will work with different containers.

To see how this works, all you need to do is change the registration of the `LifetimeDemoService` to use the `AddScoped` method:

```
builder.Services.AddScoped<LifetimeDemoService>();
```

Then change the Razor page to reference the scoped service values:

```
<p>The first scoped service returned @Model.Value</p>
<p>The second scoped service returned @Model.SecondValue</p>
```

Now when you run the application, you should see that both services produce the same value. Once the `LifetimeDemoService` is instantiated, it is reused wherever it is needed within the scope of the HTTP request. It is similar, in that respect, to a singleton, scoped to a request, rather than the application's lifetime.

Scoped lifetimes are most suitable for services that are expensive to instantiate and/or need to maintain state during a request. In a Razor Pages application, one of the most frequently used services that benefits from a scoped lifetime is the Entity Framework `DbContext`, which we will look at in more detail in the next chapter. The `DbContext` meets both criteria in that it is expensive to instantiate because it creates a connection to an external resource (the database), and it may need to maintain information about data that it has retrieved from the database.

7.2.3 Captive dependencies

When choosing a lifetime for your service, you should also take into account the lifetime of any dependency the service relies on. For example, if the